Homework Aula 2:

Exercício 1) Escreva um programa que, dado uma sequência de números inteiros gerados aleatoriamente (utilizando um while), distribua esses números em duas filas circulares distintas da seguinte forma:

- Números positivos devem ser inseridos em uma fila chamada `positivos`;
- Números negativos devem ser inseridos em uma fila chamada `negativos`;
- Números iguais a zero não devem ser armazenados, mas devem ser contabilizados.

Ao final da geração dos números, o programa deve: imprimir o conteúdo da fila de positivos; imprimir o conteúdo da fila de negativos; e exibir a quantidade total de zeros gerados. Para resolver este exercício, utilize exclusivamente as operações definidas para a interface de fila circular (create, enqueue, dequeue, empty, front, print, etc.). (ou seja, suponha que você não tenha acesso ao código fonte que implementa a fila circular, portanto não use o operador $\rightarrow$ para acessar os elementos da fila diretamente, se precisar de algum elemento use front, enqueue, dequeue, ...).

Exercício 2) Qual a complexidade, utilizando a notação assintótica $\mathcal{O}(?)$, para as seguintes operações básicas de uma estrutura de dados **fila circular** codificada através de um vetor?

	Melhor caso	Pior caso
enqueue		
dequeue		
front		
empty		
busca		

Exercício 3) Problema de '**Josephus**': a história conta que na revolta judaica contra Roma, Josephus e mais trinta e nove de seus companheiros resistiram aos romanos em uma caverna. Com a derrota iminente, eles decidiram que prefeririam morrer a serem escravos dos romanos. Eles se organizaram em um círculo. Um homem foi designado como número um, outro como dois, e assim por diante, e percorrendo o círculo no sentido horário e contando eles matavam todo o sétimo homem (após a morte a contagem reiniciava no próximo homem após aquele que foi morto). Josephus, de acordo com a lenda, era entre outras coisas um matemático realizado. Ele instantaneamente descobriu onde deveria sentar-se para ser o último a ser executado. Quando chegou a hora, em vez de matar-se, ele se juntou ao lado romano. Você pode encontrar muitas versões diferentes desta história. Algumas

versões matam cada terceiro homem e outras permitem que o último homem fuja em um cavalo. Em qualquer caso, a ideia é a mesma. Existe uma brincadeira infantil conhecida como '**batata quente**' que segue a mesma lógica do conto acima.

Supondo que $n = 5$ (cinco pessoas) e $m = 3$ (passo da morte), então a sequência de eliminações é dado por: 3, 1, 5, 2, 4 — tal que o último elemento da sequência (4) é o sobrevivente — pois

```
 1   2   3   4   5
 1   2  *3   4   5  (1 iteração - elimina o 3)
*1   2  *3   4   5  (2 iteração - elimina o 1)
*1   2  *3   4  *5  (3 iteração - elimina o 5)
*1  *2  *3   4  *5  (4 iteração - elimina o 2)
*1  *2  *3  *4  *5  (5 iteração - elimina o 4)
```

Para resolver este exercício utilize as operações definidas para a interface da fila circular (create, enqueue, dequeue, empty, front, ...). **Não** acesse os elementos armazenados na fila senão pelas operações (funções) definidas pela interface (ou seja, suponha que você não tenha acesso ao código fonte que implementa a fila circular, portanto não use o operador $\rightarrow$ para acessar os elementos da fila diretamente, se precisar de algum elemento use front, enqueue, dequeue, ...).

Exercício 4) Máquinas caça-níqueis são os principais geradores de receita para quase todos os cassinos físicos e on-line. Os componentes básicos de uma máquina caça-níqueis são:

- Moeda de caça-níquel: lugar onde você insere dinheiro ou cartões de crédito;
- Carretéis: rolo giratório com diversas imagens que formam combinações;
- Alavanca: ao puxá-la, os carretéis giram;
- Pagamento ativo: determina o valor do pagamento;

Os carretéis são rolos com grupos de símbolos que, na maioria das vezes, giram no sentido vertical. A figura abaixo ilustra uma dessas máquinas.

Sua tarefa nesse exercício é simular o movimento de 3 carretéis que contém a combinação de números de 1 até 9 em cada um deles. Os carretéis em movimento devem ser simulados através de uma fila, onde o giro aleatório para cada um dos carretéis pode ser simulado ao desinfileirar e enfileirar novamente os elementos na fila. A combinação dos números a cada sequência de giros indica se o jogador ganhou ou não. Considere como os números sorteados aqueles que ficam no início da fila após o giro aleatório em cada fila. O jogador ganha caso os 3 números que estiverem no início de cada uma das filas forem iguais. Por exemplo:

```
5 4 1
7 3 2
1 2 4
9 1 4
4 3 5
7 7 7 -> Ganhou
```

A parte de recompensa não precisa ser implementada, apenas os sorteios e lógica para decidir se ganhou ou se o jogo continua.

Utilize uma fila circular e operações definidas pela interface para resolver este problema.